

LAPORAN TUGAS BESAR STRUKTUR DATA
“Sistem Manajemen Fasilitas Kampus”

KELOMPOK 7



Disusun Oleh :

1. Alvin Aldino Rahmatullah (103112430283)
2. Gustaf Adiyatma Alfito (103112400266)
3. Muhammad Haidar Amanullah (103112400262)

Program Studi Informatika

Telkom University

Semester Ganjil 2025/2026

1. PENDAHULUAN

1.1. Latar Belakang

Pengelolaan fasilitas kampus merupakan bagian penting dalam menunjang kelancaran kegiatan akademik maupun non-akademik. Fasilitas seperti ruangan kelas, laboratorium, dan alat pendukung harus dikelola dengan baik agar dapat digunakan secara efektif dan efisien. Namun, pada prakteknya masih banyak pengelolaan fasilitas kampus yang dilakukan secara manual atau tidak terintegrasi, sehingga sering menimbulkan berbagai permasalahan.

Permasalahan yang sering muncul antara lain pencatatan data peminjaman yang tidak rapi, terjadinya bentrok jadwal penggunaan ruangan, sulitnya memantau status peminjaman, serta kurangnya kontrol terhadap hak akses pengguna. Selain itu, proses administrasi yang manual juga berpotensi menimbulkan kesalahan data dan keterlambatan informasi.

Oleh karena itu, dibutuhkan sebuah Sistem Manajemen Fasilitas Kampus yang terkomputerisasi untuk membantu proses pengelolaan data peminjaman secara terstruktur. Sistem ini dirancang berbasis bahasa pemrograman C++ dengan penyimpanan data menggunakan file CSV, serta dilengkapi dengan fitur login, registrasi, manajemen user, peminjaman ruangan, dan pengelolaan hak akses berdasarkan peran pengguna dengan memanfaatkan struktur data dan mekanisme kontrol akses berbasis peran.

1.2. Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah dalam pembuatan sistem ini adalah:

1. Bagaimana merancang sistem peminjaman fasilitas kampus yang terkomputerisasi?
2. Bagaimana mengelola data peminjaman dan data pengguna secara terstruktur dan aman?
3. Bagaimana menerapkan pembagian hak akses antara User, Admin, dan Super Admin dalam satu sistem?
4. Bagaimana meminimalkan konflik penggunaan fasilitas kampus melalui sistem yang dibangun?

1.3. Tujuan Penelitian dan Tujuan Pembuatan Sistem

Tujuan dari pembuatan Sistem Manajemen Fasilitas Kampus ini adalah:

1. Merancang dan mengimplementasikan sistem peminjaman fasilitas kampus berbasis console menggunakan bahasa C++.
2. Menerapkan sistem autentikasi dan manajemen pengguna berdasarkan peran (role).
3. Mengelola data peminjaman fasilitas kampus secara terstruktur menggunakan file CSV.
4. Mengurangi kesalahan dan konflik dalam peminjaman fasilitas kampus.

1.4 Manfaat Penelitian / Manfaat Sistem

Manfaat dari sistem yang dibangun antara lain:

1. **Bagi Pengguna**

Mempermudah proses peminjaman fasilitas kampus serta penambahan alat pendukung sesuai kebutuhan.

2. **Bagi Pengelola Kampus (Admin & Super Admin)**

Memudahkan pengawasan, pengelolaan, dan pengendalian data peminjaman serta data pengguna.

3. **Bagi Kami**

Menambah pemahaman dan pengalaman dalam menerapkan konsep pemrograman C++, struktur data, serta file handling.

2. LANDASAN TEORI

2.1. Bahasa Pemrograman

Selain bahasa pemrograman yang diwajibkan pada semester ini, C++ juga merupakan bahasa pemrograman tingkat tinggi yang mendukung paradigma pemrograman prosedural dan berorientasi objek. Bahasa ini memiliki performa yang baik, efisien dalam penggunaan memori, serta banyak digunakan dalam pengembangan aplikasi sistem. C++ dipilih dalam pembuatan sistem ini karena mendukung pengelolaan struktur data, file handling, serta logika program yang kompleks.

2.2. Sistem Informasi

Sistem informasi adalah suatu sistem yang terdiri dari komponen-komponen yang saling berinteraksi untuk mengumpulkan, mengolah, menyimpan, dan menyajikan informasi. Sistem informasi berperan penting dalam membantu pengambilan keputusan dan pengelolaan data secara terorganisir dan efisien.

2.3. Sistem Manajemen Fasilitas

Sistem Manajemen Fasilitas merupakan sistem informasi yang digunakan untuk mengelola penggunaan fasilitas secara terstruktur. Sistem ini mencakup proses pencatatan, pemantauan, dan pengendalian penggunaan fasilitas agar dapat dimanfaatkan secara optimal dan terhindar dari konflik penggunaan.

2.4. Pemakaian Struktur Data

Struktur data yang digunakan dalam sistem ini antara lain:

- **Struct** untuk menyimpan data user dan peminjaman.
- **Vector** untuk menyimpan kumpulan data yang bersifat dinamis.
- **Binary Search Tree (BST)** untuk mengelola level role pada Super Admin.
- **Array dan string** untuk pengolahan data sementara dari file CSV.

3. ANALISIS DAN PERANCANGAN SISTEM

3.1. Analisis Kebutuhan Sistem

Sistem membutuhkan:

- Input data user, data peminjaman, dan data alat tambahan.
- Proses validasi login, pengecekan ketersediaan ruangan, dan pengelolaan hak akses.
- Output berupa informasi peminjaman dan data pengguna.

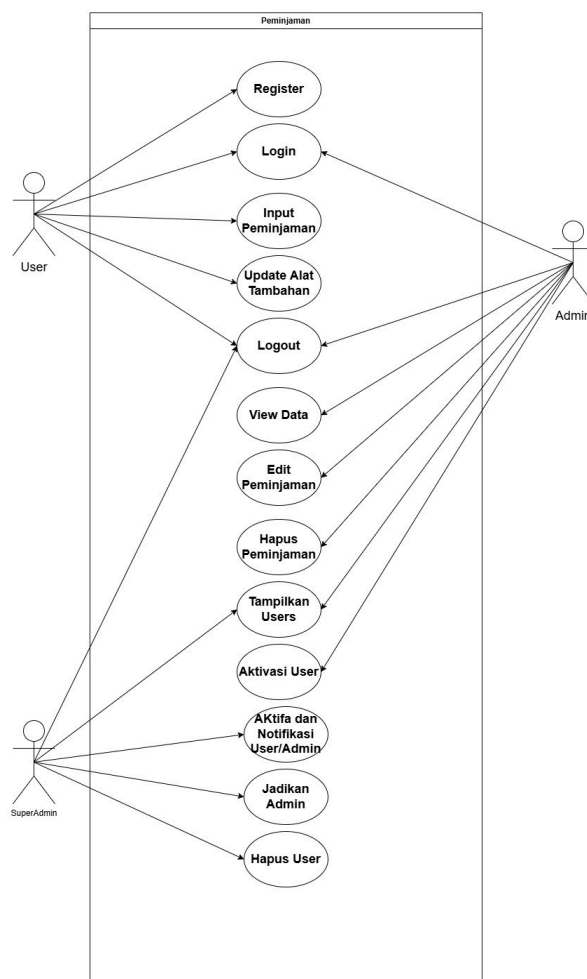
3.2. Analisis Pengguna (User, Admin, Super Admin)

1. **User** : Melakukan peminjaman ruangan dan update alat tambahan.
2. **Admin** : Mengelola data peminjaman dan melakukan aktivasi user.
3. **Super Admin** : Mengelola seluruh data user, status akun, dan role pengguna.

3.3. Perancangan Alur Sistem

Alur sistem dimulai dari proses login atau registrasi user. Setelah berhasil login, sistem akan menampilkan menu sesuai dengan role pengguna. Setiap aksi yang dilakukan akan diproses dan disimpan ke dalam file CSV.

Berikut perancangan berupa Use Case :



3.4. Fitur Pendukung

- Autentikasi login
- Registrasi user
- Aktivasi akun
- Peminjaman ruangan
- Penambahan alat tambahan
- Manajemen user dan role
- Menampilkan jumlah user

4. IMPLEMENTASI DAN PENGUJIAN

4.1. Implementasi Sistem

- sistem.h

Berisi deklarasi fungsi dan struktur data yang digunakan bersama.

```
#ifndef SISTEM_H
#define SISTEM_H
#include <string>

using namespace std;

struct User {
    int id;
    string username;
    string password;
    string role;
    string status;
};

struct Peminjaman {
    int id;
    string username;
    string ruangan;
    string hari;
    string jam;
    string alat;
    string lokasi;
};

struct RoleNode {
    string role;
    int level;
    RoleNode* left;
    RoleNode* right;
};
```

```

    RoleNode(string r, int l);
};

RoleNode* buildRoleTree();
int getRoleLevel(RoleNode* root, const string& role);

void superAdminMenu(const User& currentUser);

void login();
void registerUser();
void adminMenu();
void userMenu();
void countdown(int seconds);
void aktivasiUser();
void tampilkanUsers();

void daftarPeminjam();
void editPeminjaman();
void lokasiPeminjaman();
void alatTambahan();

#endif

```

-adminAktifasi.cpp

Kode yang kamu kirim ini berfungsi untuk manajemen akun user oleh admin, khususnya menampilkan daftar user dan mengaktifkan / menonaktifkan akun user yang tersimpan di file CSV. Berikut penjelasan gunanya secara jelas dan ringkas, tapi tetap lengkap.

```

#include "sistem.h"
#include "iostream"
#include <fstream>
#include <sstream>
#include <vector>
#include <iomanip>

using namespace std;

void tampilkanUsers(){
    ifstream file("database/Users.csv");
    string line, temp;

```

```

if (!file.is_open()) {
    cout << "Gagal membuka database user.\n";
    return;
}

getline(file, line);

cout << "\n===== DAFTAR USER =====\n";
cout << left
    << setw(5) << "ID"
    << setw(15) << "Username"
    << setw(15) << "Role"
    << setw(10) << "Status" << endl;

while (getline(file, line)){
    stringstream ss(line);
    User user;

    getline(ss, temp, ',');
    user.id = stoi(temp);
    getline(ss, user.username, ',');
    getline(ss, user.password, ',');
    getline(ss, user.role, ',');
    getline(ss, user.status, ',');

    cout << left
        << setw(5) << user.id
        << setw(15) << user.username
        << setw(15) << user.role
        << setw(10) << user.status << endl;
    }
    file.close();
}

void aktivasiUser() {
    vector<User> daftarUser;
    ifstream file("database/Users.csv");
    string line, temp;

    if (!file.is_open()) {
        cout << "Gagal membuka database user.\n";
    }
}

```

```

        return;
    }

    getline(file, line);

    while (getline(file, line)) {
        stringstream ss(line);
        User user;

        getline(ss, temp, ',');
        user.id = stoi(temp);
        getline(ss, user.username, ',');
        getline(ss, user.password, ',');
        getline(ss, user.role, ',');
        getline(ss, user.status, ',');

        daftarUser.push_back(user);
    }

    file.close();

    tampilkanUsers();

    int idDipilih;
    cout << "\nMasukkan ID User: ";
    cin >> idDipilih;

    bool ditemukan = false;

    for (auto &user : daftarUser) {
        if (user.id == idDipilih) {
            ditemukan = true;

            if (user.role == "admin" || user.role == "superadmin") {
                cout << "Admin tidak boleh mengubah status admin lain.\n";
                return;
            }

            cout << "Status saat ini: " << user.status << endl;
            cout << "Ubah status menjadi (active/inactive): ";
            cin >> user.status;
            if (user.status != "active" && user.status != "inactive") {

```



```

        cout << "Status tidak valid. Gunakan 'active' atau 'inactive'.\n";
        return;
    }
    break;
}
}

if (!ditemukan) {
    cout << "User tidak ditemukan.\n";
    return;
}

ofstream outFile("database/Users.csv");
outFile << "id;username;password;role;status\n";

for (const auto &user : daftarUser) {
    outFile << user.id << ";"
        << user.username << ";"
        << user.password << ";"
        << user.role << ";"
        << user.status << "\n";
}

outFile.close();

cout << "\nStatus user berhasil diperbarui.\n";
}

```

-login.cpp

File berfungsi untuk proses autentikasi (login) user. File ini:

- Membaca data user dari database
- Memvalidasi username dan password
- Mengecek status akun (active / inactive)
- Mengembalikan role user yang berhasil login (user/admin /superadmin)

```

#include "sistem.h"
#include "iostream"
#include "string"
#include "thread"
#include "chrono"
#include "fstream"

```

```

#include "sstream"

using namespace std;

bool authenticateUser(
    const string& inputUsername,
    const string& inputPassword,
    User& currentUser
) {
    ifstream file("database/Users.csv");
    if (!file.is_open()) {
        cout << "Gagal membuka database user.\n";
        return false;
    }

    string line;
    getline(file, line);

    while (getline(file, line)) {
        stringstream stream(line);
        string temp;

        User user;

        getline(stream, temp, ',');
        user.id = stoi(temp);

        getline(stream, user.username, ',');
        getline(stream, user.password, ',');
        getline(stream, user.role, ',');
        getline(stream, user.status, ',');

        user.username.erase(
            user.username.find_last_not_of(" \n\r\t") + 1
        );

        if (user.username == inputUsername &&
            user.password == inputPassword) {

            if (user.status != "active") {
                cout << "Akun belum aktif, hubungi admin.\n";
                return false;
            }
        }
    }
}

```

```

    }

    currentUser = user;
    return true;
}
}
return false;
}

void countdown(int detik) {
    cout << endl;
    for (int i = detik; i > 0; --i)
    {
        cout << "Silahkan Tunggu Selama " << i << " Detik..." << endl;
        this_thread::sleep_for(chrono::seconds(1));
    }
    cout << endl;
}

void login() {
    User currentUser;
    string username, password;
    int percobaan = 0;
    const int maksimal = 3;
    const int berhenti = 10;

    while (true) {
        cout << endl;
        cout << "Username: ";
        cin >> username;

        cout << "Password: ";
        cin >> password;

        if (authenticateUser(username, password, currentUser)) {
            cout << "Login berhasil sebagai " << currentUser.role << endl;
            if (currentUser.role == "admin") {
                adminMenu();
            } else if (currentUser.role == "user") {
                userMenu();
            }
        }
    }
}

```

```

    } else if (currentUser.role == "superadmin") {
        superAdminMenu(currentUser);
    }
}
else {
    percobaan++;
    cout << "-[login gagal! percobaan ke " << percobaan << "]" << endl;

    if (percobaan == maksimal) {
        cout << endl;
        cout << "--TERLALU BANYAK PERCOBAAN GAGAL--" << endl;
        cout << "--AKSES LOGIN DIBLOKIR SEMENTARA--" << endl;

        countdown(berhenti);
        percobaan = 0;
        cout << "--SILAHKAN MENCoba LOGIN KEMBALI--" << endl;
    }
}
}
}
}

```

-main.cpp

Fungsinya sebagai pengendali utama (entry point) program.

Tugas utamanya:

- Menjalankan proses login
- Mengecek role user
- Mengarahkan user ke menu yang sesuai
- Mengatur alur program secara keseluruhan

```

#include "sistem.h"
#include "iostream"
using namespace std;

int main() {
    int pilihan;

    while (true) {
        cout << "===== " << endl;
        cout << "    SELAMAT DATANG " << endl;
        cout << "===== " << endl;
        cout << "1. Login" << endl;
    }
}

```

```

        cout << "2. Register" << endl;
        cout << "3. Exit" << endl;
        cout << "Pilih: ";
        cin >> pilihan;

        switch (pilihan) {
            case 1:
                login();
                break;
            case 2:
                registerUser();
                break;
            case 3:
                return 0;
            default:
                cout << "Pilihan tidak valid!" << endl;
        }
    }
}

void adminMenu(){
    int pilih;
    do {
        cout << "===== " << endl;
        cout << " SELAMAT DATANG MENU ADMIN " << endl;
        cout << "===== " << endl;
        cout << "1. Daftar Peminjam" << endl;
        cout << "2. Edit Peminjaman" << endl;
        cout << "3. Aktivasi" << endl;
        cout << "4. Logout" << endl;
        cout << "Pilih: ";
        cin >> pilih;

        switch (pilih){
            case 1:
                daftarPeminjam();
                break;
            case 2:
                editPeminjaman();
                break;
            case 3:
                aktivasiUser();

```

```

        break;
    case 4:
        cout << "Logout berhasil." << endl;
        break;
    }
} while (pilih != 4);
}

void userMenu(){
    int pilih;
    do {
        cout << "===== " << endl;
        cout << "  SELAMAT DATANG MENU USER  " << endl;
        cout << "===== " << endl;
        cout << "1. Lokasi Peminjaman" << endl;
        cout << "2. Alat Tambahan" << endl;
        cout << "3. Logout" << endl;
        cout << "Pilih: ";
        cin >> pilih;

        switch (pilih){
            case 1:
                lokasiPeminjaman();
                break;
            case 2:
                alatTambahan();
                break;
            case 3:
                cout << "Logout berhasil." << endl;
                break;
        }
    } while (pilih != 3);
}

```

-register.cpp

Berfungsi untuk mendaftarkan user baru ke dalam sistem dengan:

- Menyimpan data ke database
- Memberikan ID otomatis
- Menetapkan role default = user
- Menetapkan status default = inactive (harus diaktifkan admin)

```
#include "sistem.h"
#include "iostream"
#include <fstream>
#include <sstream>

using namespace std;
string line, temp;

int getLastUserId(){
    ifstream file("database/Users.csv");
    int lastId = 0;

    getline(file, line);

    while (getline(file, line))
    {
        stringstream ss(line);
        getline(ss, temp, ',');
        lastId = stoi(temp);
    }
    return lastId;
}

bool cekUsername(const string &username) {
    ifstream file("database/Users.csv");
    getline(file, line);

    while (getline(file, line))
    {
        stringstream ss(line);
        User user;

        getline(ss, temp, ',');
        getline(ss, user.username, ',');
        if (user.username == username)
        {
            return true;
        }
    }
    return false;
}
```

```

void registerUser(){
    string username, password;
    cout << "=== Registrasi User Baru ===" << endl;

    while (true){
        cout << "Username: ";
        cin >> username;
        if (cekUsername(username)){
            cout << "Username sudah digunakan. Silahkan pilih nama lain.\n";
        }
        else{
            break;
        }
    }
    cout << "Password: ";
    cin >> password;

    int newId = getLastUserId() + 1;

    ofstream file("database/Users.csv", ios::app);
    if (!file.is_open()){
        cout << "Gagal membuka database.\n";
        return;
    }

    file << newId << ";" << username << ";" << password << ";user;inactive\n";
    file.close();

    cout << "\nRegistrasi berhasil!\n";
    cout << "Akun menunggu aktivasi admin.\n";
}

```

-sistemAdmin.cpp

Berisi menu khusus admin / superadmin untuk:

- Mengelola data buku
- Mengelola akun user (lihat & aktivasi)
- Menjadi pusat kontrol sistem

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include "sistem.h"

```



```

using namespace std;

void daftarPeminjam() {
    ifstream file("database/Peminjaman.csv");

    if (!file.is_open()) {
        cout << "Gagal membuka database peminjaman.\n";
        return;
    }

    string line;
    getline(file, line); // lewati header

    cout << "\n===== DAFTAR PEMINJAMAN =====\n";

    bool adaData = false;

    while (getline(file, line)) {
        stringstream ss(line);
        string temp;
        Peminjaman p;

        getline(ss, temp, ',');
        p.id = stoi(temp);

        getline(ss, p.username, ',');
        getline(ss, p.ruangan, ',');
        getline(ss, p.hari, ',');
        getline(ss, p.jam, ',');
        getline(ss, p.alat, ',');
        getline(ss, p.lokasi, ',');

        cout << "ID      : " << p.id << endl;
        cout << "User    : " << p.username << endl;
        cout << "Ruangan : " << p.ruangan << endl;
        cout << "Hari    : " << p.hari << endl;
        cout << "Jam     : " << p.jam << endl;
        cout << "Alat    : " << p.alat << endl;
        cout << "Lokasi  : " << p.lokasi << endl;
        cout << "-----\n";
    }
}

```

```

        adaData = true;
    }

    if (!adaData) {
        cout << "Belum ada data peminjaman.\n";
    }

    file.close();
}

void editPeminjaman() {
    ifstream file("database/Peminjaman.csv");

    if (!file.is_open()) {
        cout << "Gagal membuka database peminjaman.\n";
        return;
    }

    vector<Peminjaman> data;
    string line;
    getline(file, line); // header

    while (getline(file, line)) {
        stringstream ss(line);
        string temp;
        Peminjaman p;

        getline(ss, temp, ',');
        p.id = stoi(temp);

        getline(ss, p.username, ',');
        getline(ss, p.ruangan, ',');
        getline(ss, p.hari, ',');
        getline(ss, p.jam, ',');
        getline(ss, p.alat, ',');
        getline(ss, p.lokasi, ',');

        data.push_back(p);
    }
    file.close();

    if (data.empty()) {

```

```

    cout << "Tidak ada data peminjaman.\n";
    return;
}

cout << "\n===== DATA PEMINJAMAN =====\n";
for (const auto& p : data) {
    cout << "ID: " << p.id
        << " | User: " << p.username
        << " | Ruang: " << p.ruangan
        << " | Hari: " << p.hari
        << " | Jam: " << p.jam << endl;
}

int idCari;
cout << "\nMasukkan ID peminjaman: ";
cin >> idCari;

int aksi;
cout << "1. Edit Peminjaman\n";
cout << "2. Hapus Peminjaman\n";
cout << "Pilih aksi: ";
cin >> aksi;
cin.ignore();

bool ditemukan = false;

for (auto it = data.begin(); it != data.end(); ++it) {
    if (it->id == idCari) {
        ditemukan = true;

        if (aksi == 1) {
            cout << "\n=== INPUT DATA BARU ===\n";
            cout << "Ruangan : ";
            getline(cin, it->ruangan);

            cout << "Hari   : ";
            getline(cin, it->hari);

            cout << "Jam    : ";
            getline(cin, it->jam);

            cout << "Alat   : ";

```

```

        getline(cin, it->alat);

        cout << "Lokasi : ";
        getline(cin, it->lokasi);
    }
    else if (aksi == 2) {
        data.erase(it);
        cout << "\nData peminjaman BERHASIL dihapus.\n";
    }
    else {
        cout << "Pilihan tidak valid.\n";
        return;
    }

    break;
}

if (!ditemukan) {
    cout << "ID tidak ditemukan.\n";
    return;
}

ofstream out("database/Peminjaman.csv");
out << "id;username;ruangan;hari;jam;alat;lokasi\n";

for (const auto& p : data) {
    out << p.id << ","
        << p.username << ","
        << p.ruangan << ","
        << p.hari << ","
        << p.jam << ","
        << p.alat << ","
        << p.lokasi << endl;
}

out.close();

if (aksi == 1) {
    cout << "\nData peminjaman BERHASIL diubah.\n";
}
}

```

-sistemUser.cpp

Berisi menu khusus user biasa, dengan akses terbatas dibanding admin. User tidak bisa mengelola user lain, hanya bisa melihat dan mencari data buku.

```
#include "sistem.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <limits>

using namespace std;

void lokasiPeminjaman() {
    ifstream file("database/Peminjaman.csv");
    if (!file.is_open()) {
        cout << "File Peminjaman.csv tidak ditemukan!\n";
        return;
    }

    string line;
    getline(file, line);

    int lastId = 0;

    while (getline(file, line)) {
        stringstream ss(line);
        string sid;
        getline(ss, sid, ',');
        lastId = stoi(sid);
    }
    file.close();

    int newId = lastId + 1;

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    Peminjaman p;
    p.id = newId;

    cout << "Username : ";
    getline(cin, p.username);
```

```
cout << "Ruangan : ";
getline(cin, p.ruangan);

cout << "Hari : ";
getline(cin, p.hari);

cout << "Jam : ";
getline(cin, p.jam);

cout << "Lokasi : ";
getline(cin, p.lokasi);

p.alat = "-";

file.open("database/Peminjaman.csv");
getline(file, line);

bool terpakai = false;
while (getline(file, line)) {
    stringstream ss(line);
    Peminjaman cek;
    string sid;

    getline(ss, sid, ',');
    getline(ss, cek.username, ',');
    getline(ss, cek.ruangan, ',');
    getline(ss, cek.hari, ',');
    getline(ss, cek.jam, ',');
    getline(ss, cek.alat, ',');
    getline(ss, cek.lokasi, ',');

    if (cek.ruangan == p.ruangan && cek.lokasi == p.lokasi) {
        terpakai = true;
        break;
    }
}
file.close();

if (terpakai) {
    cout << "Ruangan sedang dipakai.\n";
    return;
```

```

}

ofstream out("database/Peminjaman.csv", ios::app);
out << p.id << ";" << p.username << ";" << p.ruangan << ";"
    << p.hari << ";" << p.jam << ";" << p.alat << ";" << p.lokasi << endl;
out.close();

cout << "Peminjaman berhasil disimpan.\n";
cout << "ID Peminjaman: " << p.id << endl;
}

void alatTambahan() {
    ifstream file("database/Peminjaman.csv");
    if (!file.is_open()) {
        cout << "File Peminjaman.csv tidak ditemukan!\n";
        return;
    }

    string data[200];
    int n = 0;

    while (getline(file, data[n])) {
        n++;
    }
    file.close();

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    int idInput;
    cout << "Masukkan ID Peminjaman: ";
    cin >> idInput;
    cin.ignore();

    bool ditemukan = false;

    for (int i = 1; i < n; i++) {
        stringstream ss(data[i]);
        Peminjaman p;
        string sid;

        getline(ss, sid, ';');
        p.id = stoi(sid);
    }
}

```

```

getline(ss, p.username, ',');
getline(ss, p.ruangan, ',');
getline(ss, p.hari, ',');
getline(ss, p.jam, ',');
getline(ss, p.alat, ',');
getline(ss, p.lokasi, ',');

if (p.id == idInput) {
    ditemukan = true;

    cout << "\nData Peminjaman:\n";
    cout << "Username : " << p.username << endl;
    cout << "Ruangan : " << p.ruangan << endl;
    cout << "Lokasi : " << p.lokasi << endl;
    cout << "Alat lama: " << p.alat << endl;

    string alatBaru;
    cout << "\nMasukkan alat tambahan: ";
    getline(cin, alatBaru);

    if (p.alat == "-" || p.alat.empty()) {
        p.alat = alatBaru;
    } else {
        p.alat = p.alat + ", " + alatBaru;
    }

    data[i] = to_string(p.id) + ";" + p.username + ";" +
        p.ruangan + ";" + p.hari + ";" + p.jam + ";" +
        p.alat + ";" + p.lokasi;

    cout << "Alat tambahan berhasil ditambahkan.\n";
    break;
}

if (!ditemukan) {
    cout << "ID peminjaman tidak ditemukan.\n";
    return;
}

ofstream out("database/Peminjaman.csv");

```



```

for (int i = 0; i < n; i++) {
    out << data[i] << endl;
}
out.close();
}

```

-superAdmin.cpp

berisi menu khusus superadmin dengan hak akses paling tinggi, yaitu:

- Semua fitur admin
- Menambah dan mengelola akun admin
- Mengontrol penuh manajemen user

```

#include "sistem.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <cctype>
#include <iomanip>

using namespace std;

RoleNode::RoleNode(string r, int l)
    : role(r), level(l), left(nullptr), right(nullptr) {}

RoleNode* insertRole(RoleNode* root, const string& role, int level) {
    if (!root) return new RoleNode(role, level);
    if (level < root->level)
        root->left = insertRole(root->left, role, level);
    else
        root->right = insertRole(root->right, role, level);
    return root;
}

int getRoleLevel(RoleNode* root, const string& role) {
    if (!root) return -1;
    if (root->role == role) return root->level;

    int kiri = getRoleLevel(root->left, role);
    if (kiri != -1) return kiri;
    return getRoleLevel(root->right, role);
}

```

```

vector<User> loadUsers() {
    vector<User> users;
    ifstream file("database/Users.csv");

    if (!file.is_open()) {
        cout << "Database user tidak ditemukan.\n";
        return users;
    }

    string line;
    getline(file, line);

    while (getline(file, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        User user;
        string temp;

        getline(ss, temp, ',');
        user.id = stoi(temp);

        getline(ss, user.username, ',');
        getline(ss, user.password, ',');
        getline(ss, user.role, ',');
        getline(ss, user.status);

        users.push_back(user);
    }
    return users;
}

void saveUsers(const vector<User>& users) {
    ofstream file("database/Users.csv", ios::trunc);
    file << "id;username;password;role;status\n";
    for (const auto& u : users) {
        file << u.id << "," << u.username << ","
            << u.password << "," << u.role
            << "," << u.status << "\n";
    }
}

```

```

}

void tampilkanUser() {
    auto users = loadUsers();

    cout << "\n===== DAFTAR USER =====\n";
    cout << left
        << setw(5) << "ID"
        << setw(15) << "Username"
        << setw(15) << "Role"
        << setw(10) << "Status" << endl;
    for (const auto& u : users) {
        cout << left
            << setw(5) << u.id
            << setw(15) << u.username
            << setw(15) << u.role
            << setw(10) << u.status << endl;
    }
}

void ubahStatusUser() {
    auto users = loadUsers();
    int id;
    cout << "ID user: ";
    cin >> id;

    for (auto& u : users) {
        if (u.id == id) {
            u.status = (u.status == "active") ? "inactive" : "active";
            saveUsers(users);
            cout << "Status berhasil diubah.\n";
            return;
        }
    }
    cout << "User tidak ditemukan.\n";
}

void jadikanAdmin() {
    auto users = loadUsers();
    int id;
    cout << "ID user: ";
    cin >> id;

```

```

    for (auto& u : users) {
        if (u.id == id) {
            u.role = "admin";
            saveUsers(users);
            cout << "User sekarang admin.\n";
            return;
        }
    }
    cout << "User tidak ditemukan.\n";
}

```

```

void hapusUser() {
    auto users = loadUsers();
    int id;
    cout << "ID user: ";
    cin >> id;

    vector<User> hasil;
    bool ketemu = false;

    for (auto& u : users) {
        if (u.id != id)
            hasil.push_back(u);
        else
            ketemu = true;
    }

    if (!ketemu) {
        cout << "User tidak ditemukan.\n";
        return;
    }

    saveUsers(hasil);
    cout << "User berhasil dihapus.\n";
}

```

```

void superAdminMenu(const User& currentUser) {
    RoleNode* root = nullptr;
    root = insertRole(root, "user", 0);
    root = insertRole(root, "admin", 1);
}

```

```

root = insertRole(root, "superadmin", 2);

if (getRoleLevel(root, currentUser.role) < 2) {
    cout << "Akses ditolak. Bukan Super Admin.\n";
    return;
}

int pilih;
do {
    cout << "===== " << endl;
    cout << " SELAMAT DATANG SUPER ADMIN " << endl;
    cout << "===== " << endl;
    cout << "1. Lihat user\n";
    cout << "2. Aktif / Nonaktif user\n";
    cout << "3. Jadikan admin\n";
    cout << "4. Hapus user\n";
    cout << "0. Keluar\n";
    cout << "Pilih: ";
    cin >> pilih;

    switch (pilih) {
        case 1: tampilkanUser(); break;
        case 2: ubahStatusUser(); break;
        case 3: jadikanAdmin(); break;
        case 4: hapusUser(); break;
        case 0: cout << "Keluar...\n"; break;
        default: cout << "Menu tidak valid.\n";
    }
} while (pilih != 0);
}

```

4.2. Pengujian Sistem

Registrasi:

1. Buat Username Baru

```

Pilih: 2
=== Registrasi User Baru ===
Username: Mulyono
Password: Mulyono123

Registrasi berhasil!
Akun menunggu aktivasi admin.

```

Di menu Registration kita akan diarahkan ke dalam forum pengisian username baru serta password baru yang akan kita gunakan untuk login. Tapi masi inactive alias harus menunggu konfirmasi admin.

User:

1.Login User

```
Username: user
Password: user123
Login berhasil sebagai user
=====
      SELAMAT DATANG MENU USER
=====
1. Lokasi Peminjaman
2. Alat Tambahan
3. Logout
Pilih:
```

User Memasukkan Username dan kata sandi untuk login tapi jika login salah terus menerus maka program memberikan waktu tunggu sampai kamu bisa akses kembali.

2.Menu Lokasi Peminjaman

```
Pilih: 1
Username : bahlil
Ruangan  : 102
Hari     : senin
Jam      : 09.00
Lokasi   : Rektorat
Peminjaman berhasil disimpan.
ID Peminjaman: 4
```

Sistem akan Menampilkan forum pengisian yang harus diisi user berupa Username,Ruangan,Hari,Jam,Lokasi dan sistem akan menampilkan pesan “Peminjaman Berhasil disimpan” dan akan diberikan ID

3.Menu Alat Tambahan

```
Masukkan ID Peminjaman: 4

Data Peminjaman:
Username : Alvin
Ruangan  : 106
Lokasi   :
Alat lama: Rektorat

Masukkan alat tambahan: Kursi
Alat tambahan berhasil ditambahkan.
```

Sistem akan meminta user memasukkan ID dan dengan data yang sudah ada, maka sistem akan meminta user memasukkan alat tambahan yang ingin dipinjam.

3. Logout

```
Pilih: 3
Logout berhasil.
```

Maka akan keluar dari program

Admin:

1.Login Admin

```
Username: admin
Password: admin123
Login berhasil sebagai admin
=====
SELAMAT DATANG MENU ADMIN
=====
1. Daftar Peminjam
2. Edit Peminjaman
3. Aktivasi
4. Logout
Pilih:
```

Sistem akan menampilkan menu daftar peminjam , edit peminjaman , aktivasi dan logout setelah admin berhasil login dan jika login salah terus menerus maka program memberikan waktu tunggu sampai kamu bisa akses kembali.

2.Menu Daftar Peminjam

```
-----
ID      : 3
User    : gustaf
Ruangan : 123
Hari    : senin
Jam     : 09-09
Alat    : -
Lokasi  : rektorat
-----
ID      : 4
User    : bahlil
Ruangan : 102
Hari    : senin
Jam     : 09.00
Alat    : -
Lokasi  : Rektorat
-----
```

Sistem akan menampilkan daftar peminjam yang berhasil booking.

3.Menu Edit Peminjam

```
Pilih: 2

===== DATA PEMINJAMAN =====
ID: 4 | User: Alvin | Ruang: 106 | Hari: Jumat | Jam: 17.00
ID: 1 | User: yamal | Ruang: 304 | Hari: senin | Jam: 17.00
ID: 2 | User: ahmad | Ruang: 405 | Hari: senin | Jam: 12.00
ID: 3 | User: gustaf | Ruang: 123 | Hari: senin | Jam: 09-09
ID: 4 | User: bahlil | Ruang: 102 | Hari: senin | Jam: 09.00

Masukkan ID peminjaman: 4
1. Edit Peminjaman
2. Hapus Peminjaman
Pilih aksi:
```

Menu Edit Peminjam yang pertama, sistem akan menampilkan edit peminjaman atau hapus peminjaman.

```
Pilih aksi: 1

=== INPUT DATA BARU ===
Ruangan : Rektorat
Hari    : Senin
Jam     : 09.00
Alat    : Meja
Lokasi  : Rektorat
```

Jika pilih menu Edit Peminjaman, maka sistem akan menampilkan menu yang sama ketika user membeking ruangan.

```
Pilih aksi: 2

Data peminjaman BERHASIL dihapus.
=====
SELAMAT DATANG MENU ADMIN
=====
```

Ketika memiliki menu Hapus Peminjaman, maka sistem akan menghapus peminjam melalui ID yang dipilih di awal.

4. Aktivasi

```
Pilih: 3

===== DAFTAR USER =====
ID  Username   Role    Status
1   admin      admin   active
2   user       user    active
3   gustaf     user    inactive

Masukkan ID User: 3
Status saat ini: inactive
Ubah status menjadi (active/inactive): active

Status user berhasil diperbarui.
```

Di menu aktivasi kita bisa memperbarui status dari orang yang sudah registrasi atau yang baru registrasi.

5. Logout

```
Pilih: 4
Logout berhasil.
```

Maka akan keluar dari program

4.3. Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem manajemen user dan Admin berjalan dengan baik, seluruh fitur berfungsi sesuai kebutuhan, serta pembagian hak akses antara user, admin, dan superadmin telah diterapkan dengan benar. Sistem juga berhasil mencegah akses tidak sah melalui validasi login dan status akun

5. PENUTUP

5.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem Aplikasi Manajemen Fasilitas Kampus yang dibangun telah berjalan dengan baik dan sesuai dengan tujuan yang diharapkan. Sistem ini mampu mengelola proses login, registrasi, serta pembagian hak akses antara user, admin, dan superadmin secara jelas dan terstruktur.

Sistem berhasil menerapkan mekanisme autentikasi dan otorisasi berbasis role, sehingga setiap pengguna hanya dapat mengakses fitur sesuai dengan kewenangannya. Penggunaan file CSV sebagai media penyimpanan data terbukti mampu menyimpan dan mengelola data user secara efektif untuk kebutuhan sistem berskala kecil hingga menengah.

Selain itu, hasil pengujian menunjukkan bahwa seluruh fitur utama seperti login, registrasi, aktivasi user, manajemen fasilitas kampus, serta pengelolaan admin dapat berfungsi dengan benar tanpa kesalahan yang berarti. Dengan demikian, sistem ini dinilai layak digunakan sebagai aplikasi manajemen sederhana dan dapat dikembangkan lebih lanjut, misalnya dengan penambahan enkripsi password, penggunaan database

relasional, atau antarmuka berbasis grafis untuk meningkatkan keamanan dan kenyamanan pengguna.

5.2. Tautan

1. GitHub : [Klik Untuk Melihat](#)
2. Power Point : [Klik Untuk Melihat](#)